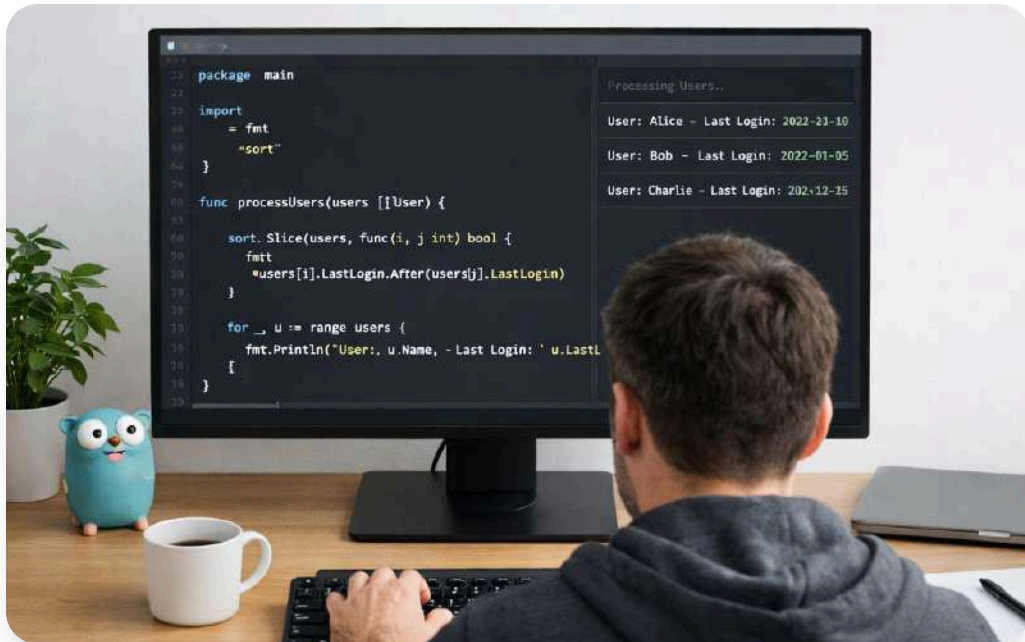




Golang Backend Pertemuan 2

Dasar percabangan dan perulangan Go untuk logika backend yang rapi dan mudah diuji.



Juara Coding

June 2026

Tujuan Pembelajaran



01 Memahami percabangan

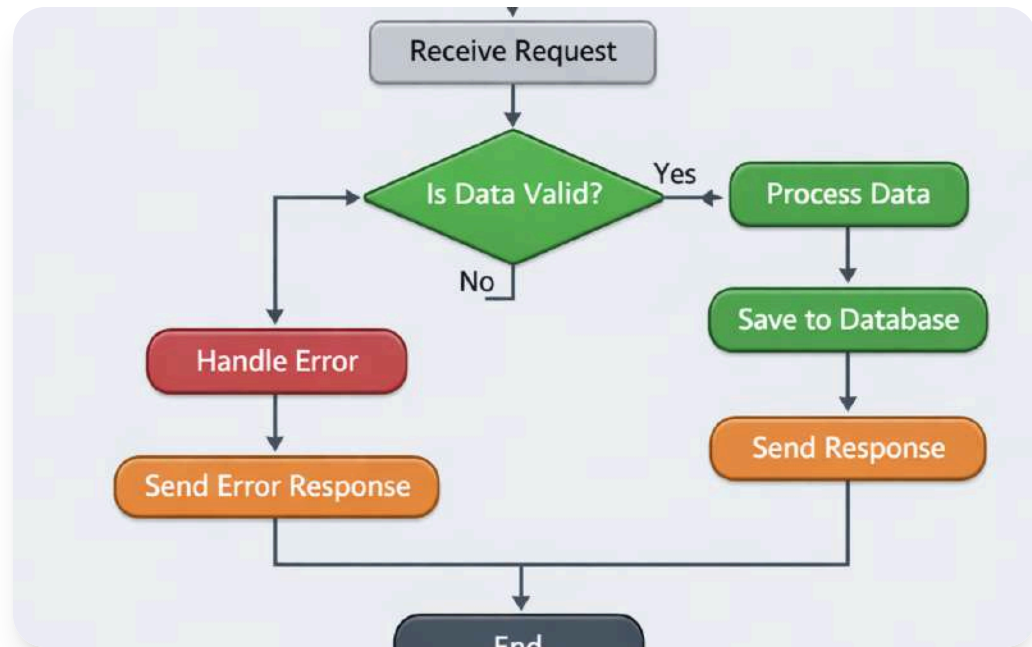
Peserta memahami penggunaan *if*, *else*, dan *switch* untuk mengatur alur logika program.

03 Menerapkan pada backend

Peserta mampu memakai logika ini untuk validasi data, cek status, dan iterasi sederhana di backend.

02 Menguasai perulangan

Peserta memahami penggunaan *for* untuk melakukan iterasi pada data atau proses berulang.



Mengapa Percabangan Penting

Di backend, percabangan membantu sistem memilih aksi sesuai kondisi, seperti data kosong, hak akses pengguna, atau validitas request.

```
var value interface{}  
  
value = nil  
  
if value == nil {  
    fmt.Println("Value is nil")  
}
```

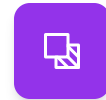
Pengenalan if pada Go

Go memakai **if** saat kondisi boolean bernilai true.

Contoh if Sederhana

```
if value == nil {  
    return None  
}  
  
// do something else...
```

Struktur *if* menjalankan aksi hanya saat syarat terpenuhi, misalnya validasi umur sebelum proses backend dilanjutkan.



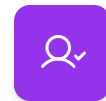
Cek Kondisi

Kode memeriksa apakah nilai *umur* lebih besar atau sama dengan 17.



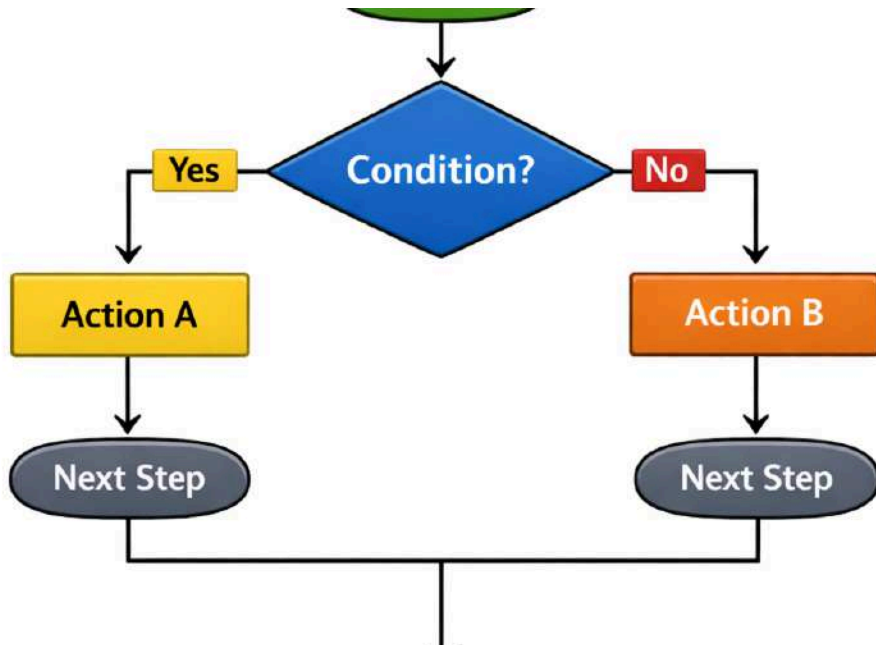
Aksi Jika Benar

Pesan ditampilkan hanya ketika kondisi bernilai benar.



Validasi Backend

Pola ini dipakai untuk memastikan syarat terpenuhi sebelum proses berikutnya.



Menggunakan if else

Gunakan *else* saat kondisi tidak terpenuhi. Struktur *if else* cocok untuk dua kemungkinan utama agar alur logika lebih jelas.

Contoh *if else*

```
if productID == 0 {  
    return "", fmt.Errorf("Invalid product ID")  
}  
  
stock, err := getStockFromDB(productID)  
if err == nil {  
    return "", err  
}  
if stock == nil {  
    return "None", nil  
}  
return "In Stock", nil  
}
```

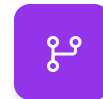
```
// Example usage  
status, err := CheckProductStock(123)  
fmt.Println(status, err)
```

Struktur *if else* dipakai untuk mengecek kondisi, seperti ketersediaan stok sebelum pesanan diproses.



Kondisi stok

Program memeriksa apakah nilai *stok* lebih besar dari nol sebelum menentukan hasilnya.



Dua kemungkinan

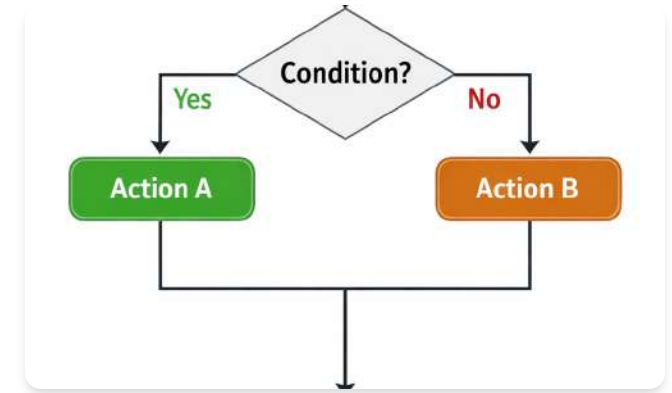
Jika stok tersedia, sistem menampilkan produk ada; jika tidak, produk dinyatakan habis.



Contoh backend

Logika ini umum dipakai saat validasi barang sebelum proses pesanan dilanjutkan.

Menggunakan *if else if else*



01 Untuk banyak kondisi

Gunakan *else if* saat jumlah kondisi lebih dari dua agar pemeriksaan bisa dilakukan secara berurutan.

02 Cek kemungkinan berurutan

Struktur ini membantu program memeriksa beberapa kemungkinan satu per satu sampai menemukan kondisi yang terpenuhi.

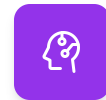
03 Eksekusi yang efisien

Setelah satu kondisi terpenuhi, program menjalankan blok tersebut lalu melewati kondisi berikutnya.

Contoh else if

```
if score >= 90 {  
    fmt.Println("Grade: Excellent")  
}  
else if score >= 75 {  
    fmt.Println("Grade: Good")  
}  
else if score >= 50 {  
    fmt.Println("Grade: Average")  
}  
else {  
    fmt.Println("Grade: Below Average")  
}  
}
```

Struktur *else if* dipakai untuk klasifikasi bertingkat, misalnya menentukan kategori nilai, prioritas, atau level akses.



Kondisi Berurutan

Program memeriksa nilai dari syarat tertinggi ke syarat berikutnya secara berurutan.



Hasil Klasifikasi

Pada contoh ini, nilai 85 memenuhi syarat kedua sehingga output yang tampil adalah B.



Cocok untuk Kategori

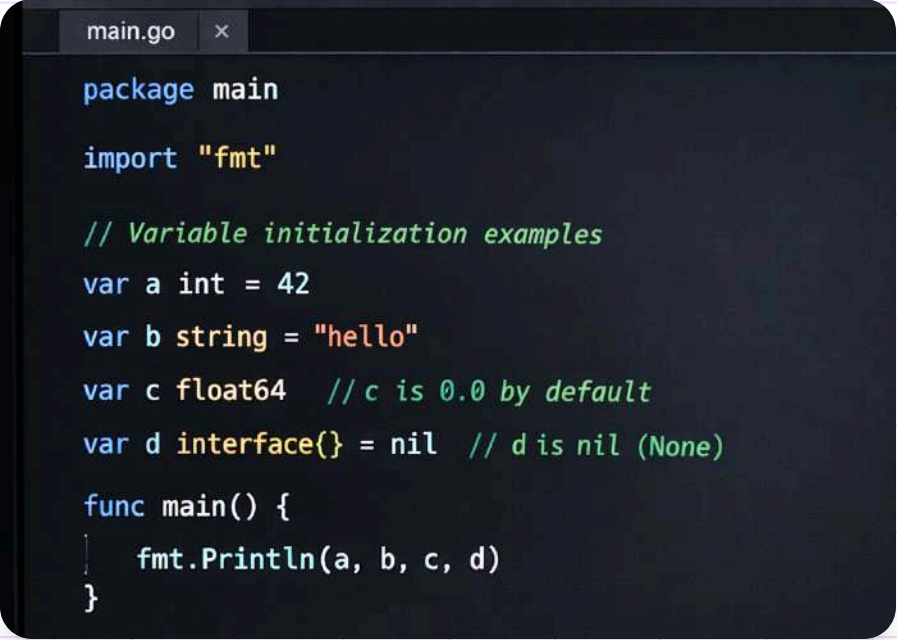
Pola ini umum digunakan untuk membagi data ke dalam level atau kategori tertentu.

```
func main() {  
    if x := 10; x > 5 {  
        y := x * 2  
        fmt.Println(y)  
    }  
}  
  
fmt.Println(x)  
fmt.Println("Done")
```

if dengan Inisialisasi

Go mendukung **if** dengan inisialisasi singkat agar kode lebih ringkas.

Contoh if dengan Inisialisasi



```
package main

import "fmt"

// Variable initialization examples
var a int = 42
var b string = "hello"
var c float64 // c is 0.0 by default
var d interface{} = nil // d is nil (None)

func main() {
    fmt.Println(a, b, c, d)
}
```

Variabel dibuat langsung di dalam *if* untuk memeriksa nilai singkat seperti panjang data, hasil parsing, atau proses backend sederhana.

01
10

Inisialisasi di if

Variabel *panjang* dibuat langsung sebelum kondisi diperiksa dalam blok *if*.



Ruang Lingkup Terbatas

Variabel hanya digunakan di area *if*, sehingga kode lebih rapi dan fokus.



Cocok untuk Backend

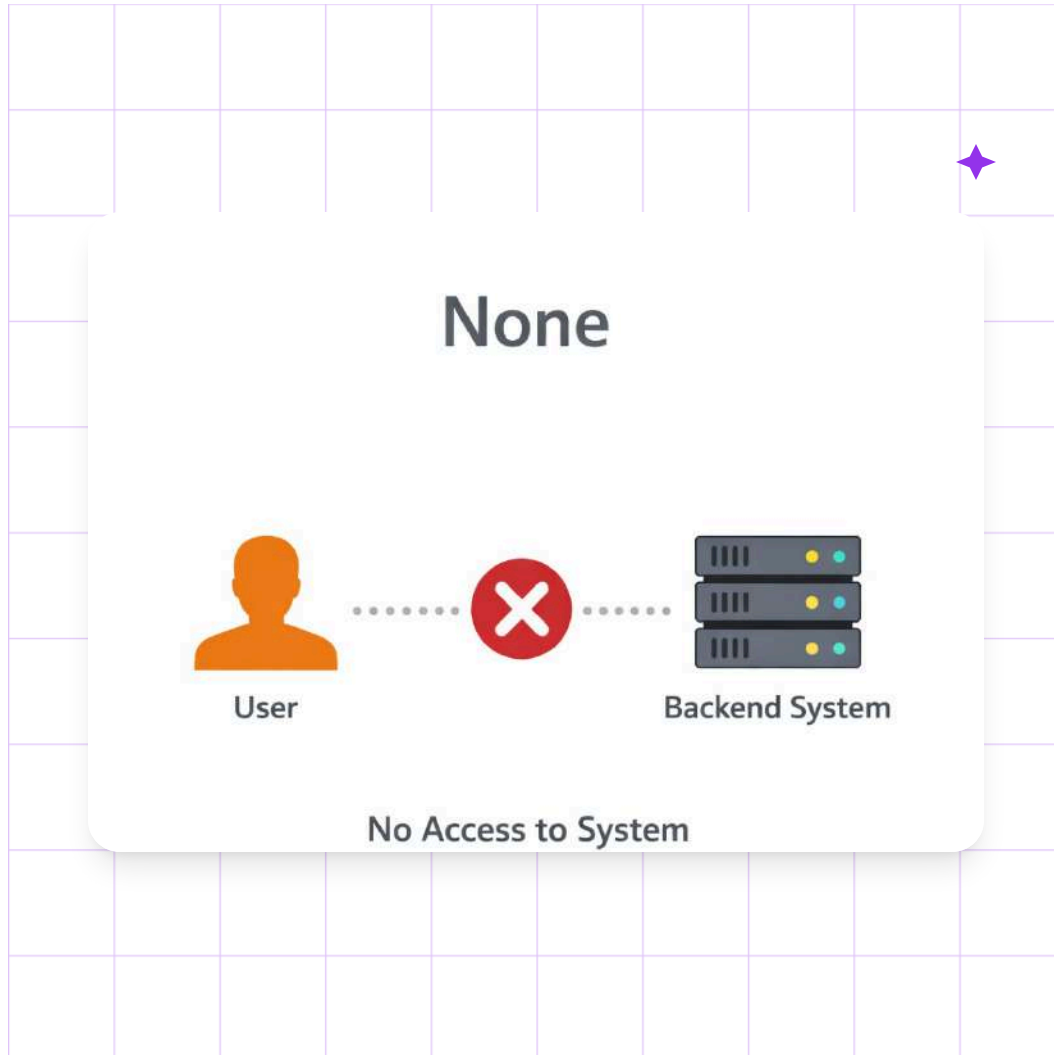
Sering dipakai saat memeriksa hasil parsing, validasi data, atau proses singkat lainnya.

```
switch code {  
  case 200:  
    return "OK".  
  case 404:  
    return "Not Found".  
  case 500:  
    return "Internal Server Error".  
  default:  
    return "Unknown Status".  
}  
}
```

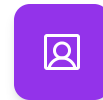
Pengenalan switch pada Go

Selain if, Go menyediakan **switch** untuk menangani banyak kondisi dengan lebih rapi, mudah dibaca, dan cocok untuk kode backend.

Contoh switch



Switch digunakan untuk menangani beberapa kondisi, misalnya menentukan hak akses pengguna di aplikasi backend.



Kasus admin

Jika role adalah admin, sistem memberikan akses penuh kepada pengguna.



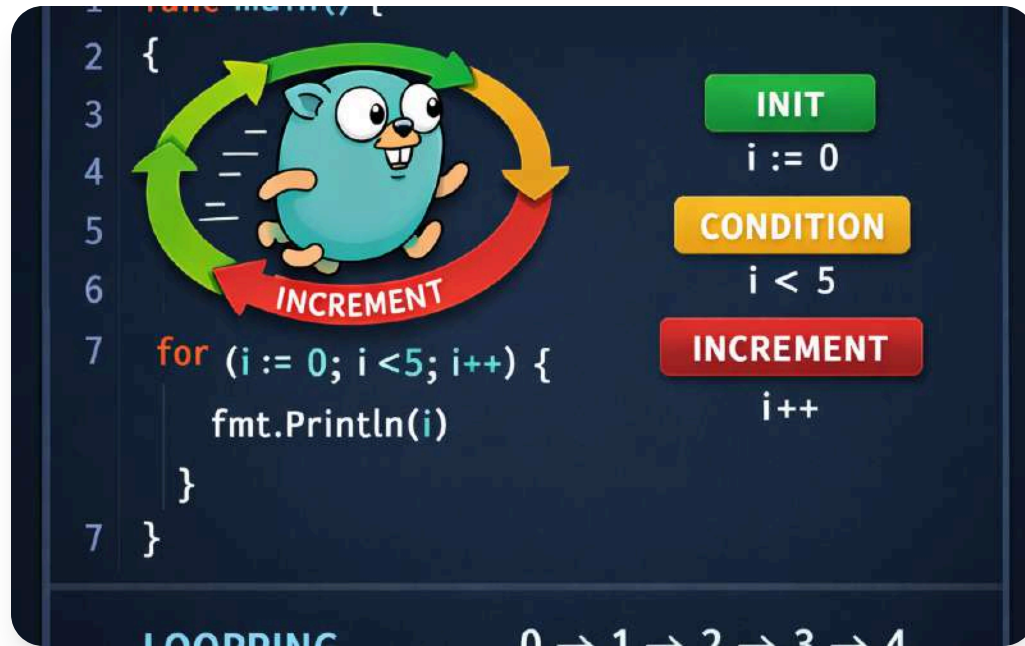
Kasus editor

Jika role adalah editor, sistem hanya memberikan hak untuk mengedit.



Default akses

Jika tidak cocok, sistem menetapkan akses terbatas sebagai perilaku default.



Pengenalan Perulangan for

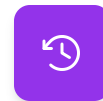
Di Go, **for** adalah perulangan utama untuk hitungan, kondisi, atau iterasi terus-menerus hingga syarat tertentu terpenuhi.

Contoh for dengan Counter

```
package main

func main() {
    for i := 0; i < 5; i++ {
        fmt.Println("Count:", i)
    }
}
```

Perulangan ini berjalan dari **1 sampai 5** dan umum dipakai untuk data berurutan, percobaan terbatas, atau akses berdasarkan indeks.



Nilai Awal dan Batas

Variabel i dimulai dari 1 dan perulangan berhenti saat nilainya lebih dari 5.



Kenaikan Otomatis

Setiap putaran, nilai i bertambah satu dengan i++ hingga kondisi tidak terpenuhi.



Penggunaan Umum

Cocok untuk memproses urutan data, uji coba terbatas, dan akses elemen via indeks.

```
2
3 import "fmt"
4
5 func main() {
6     for i := 1; i <= 5; i++ {
7         fmt.Println("Iteration:", i)
8     }
9 }
```

for sebagai while

for ini mirip **while** di bahasa lain, cocok saat jumlah perulangan belum diketahui sejak awal dan berhenti saat kondisi terpenuhi.


```
func main() {  
    data := []*int{1, 2, nil, 4, nil, 5}  
    for _, val := range data {  
        if val == nil {  
            fmt.Println("None")  
        }  
    } else {  
        fmt.Println(*val)  
    }  
}
```

Menggunakan *for range*

Go menyediakan *for range* untuk membaca array, slice, string, map, dan channel dengan mengambil index serta nilai secara praktis.

Contoh for range

```
user_list = ["Alice", "Bob", None, "Charlie", "David", "Eve"]

# Iterating through the user list
for user in user_list:
    if user is None:
        print("Skipping None value")
    else:
        print(f"User: {user}")
```

Kode ini menampilkan posisi dan isi data saat iterasi slice. Berguna untuk daftar pengguna, produk, atau item request di backend.



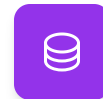
Iterasi slice

range membaca setiap elemen pada slice secara berurutan beserta posisinya.



Ambil indeks dan nilai

Variabel i berisi indeks, sedangkan user menyimpan nilai elemen saat ini.



Cocok untuk backend

Pola ini umum dipakai untuk memproses data pengguna, produk, atau item request.

break dan continue



break menghentikan loop

Dipakai untuk mengakhiri perulangan lebih cepat saat kondisi tertentu sudah terpenuhi.



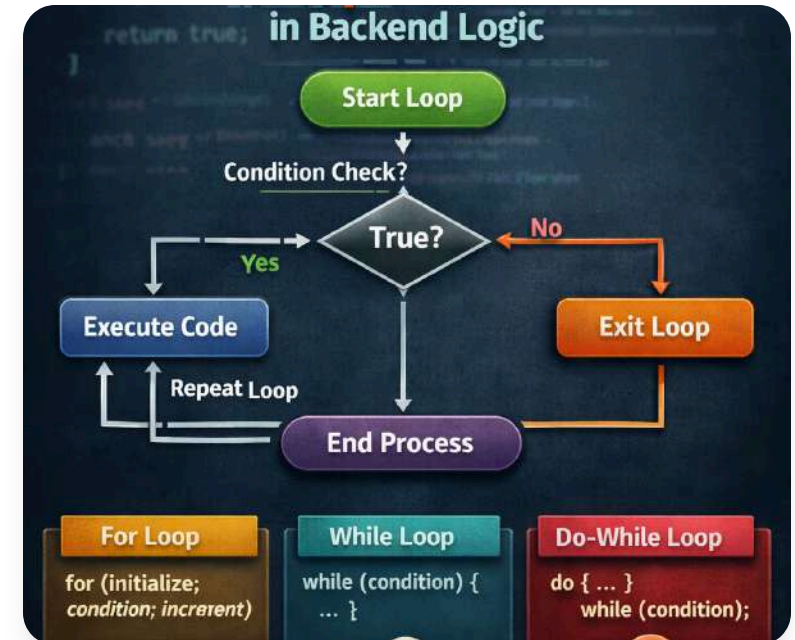
Efisien untuk backend

Membantu menyaring data atau menghentikan proses ketika kondisi penting sudah tercapai.



continue melewati iterasi

Digunakan untuk melewati satu iterasi dan langsung lanjut ke proses berikutnya.



```
return jsonify(user)
```

```
def get_user_by_id(user_id):
```

```
    query = "SELECT * FROM users WHERE id = %s"
```

```
    result = db.execute(query, (user_id,))
```

```
    return result.fetchone()
```

```
def process_order(order_id):
```

```
    order = get_order_by_id(order_id)
```

```
    if not order:
```

```
        logger.log("Order not found")
```

```
    return False
```

```
    items = get_order_items(order_id)
```

```
    total_price = calculate_total(items)
```

```
    if not charge_payment('User',
```

```
        logger.log("User", "id_user", Payypiten")
```

```
    return False
```

```
update_order_status(order_id, "completed")
```

Ringkasan



"Mempelajari percabangan dan perulangan sebagai fondasi logika backend."

Materi Pertemuan

```
def get_cached_data(key):
```

```
    data = cache.get(key)
```

```
    if None == data: # if data is None
```

```
        data = fetch_data_from_db(key)
```

```
        cache.set(key, data, timeout=3600)
```

```
    return data
```

```
try:
```

```
    process_order(order_id)
```

```
    logger.info("Order processed successfully")
```

```
except Exception as e:
```

```
    logger.error(f"Order processing failed: {e}")
```

```
    handle_error(e)
```

```
def notify_user(user_id, message):
```

```
    # Send notification to user
```

2 Soal Intermediate

01
10

Soal 1

Buat program Go untuk menentukan grade A, B, C, atau D dari nilai siswa, serta validasi nilai 0–100.



Soal 2

Buat program Go yang mencetak angka 1–20, lalu tampilkan hanya bilangan genap yang tidak habis dibagi 4.

